Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 183 (2021) 462–470

Procedia Computer Science

www.elsevier.com/locate/procedia

10th International Conference of Information and Communication Technology (ICICT-2020)

Design and implementation of EtherCAT Master based on Loongson

Changhui Wang[a,b], Liaomo Zheng[a,b,*], Beibei Li[a,b], Zeyang Li[a,b]

[a]*Shenyang Institute of Computing Technology,Chinese Academy of Sciences,Shenyang 110168 ,China*
[b]*University of Chinese Academy of Sciences,Beijing 100049,China*

Abstract

With the gradual development and competition among enterprises, the products of motion control system are diversified and the performance difference is bigger. The technology of motion control system is monopolized by a few foreign professional manufacturers. The development of motion controller in China is relatively late, and the number with independent research and development ability is still in the minority. Compared with foreign countries, some developed countries still restrict the export of high-end products to China. This paper will use Loongson 3A3000 processor to realize open motion controller. Loongson series processor is a general domestic processor developed by Institute of computing technology, Chinese Academy of Sciences. It uses MIPS instruction system. At present, Loongson is gradually catching up with and surpassing the world advanced level. The communication module of motion controller uses EtherCAT real-time Ethernet, which has the characteristics of high efficiency, flexible topology and short refresh cycle. At present, there are few cases of open motion controller based on domestic CPU Loongson, which provides a typical case for independent innovation in China.

Keywords: Loongson; Motion controller; EtherCAT; Real-time industrial Ethernet

* Corresponding author. Tel.: +86-186-0405-0334
E-mail address: zhengliaomo@sict.ac.cn

10.1016/j.procs.2021.02.085

1. Introduction

With the rapid development of electronic technology, computer technology, communication technology and automation control technology, as a core component in the field of industrial automation, motion controller plays an increasingly important role in this field. Motion controller is more and more widely used in various industries of automation equipment, such as CNC machine tools, electronic assembly automatic detection, assembly line equipment, and even in the aerospace and national defense fields. After decades of market competition, the technology of traditional high-end motion control system has been monopolized by a few professional manufacturers.

The development of motion controller has a long history. In the early stage, NCG, OMAC, TEAM API and other project plans proposed by the United States have a great impact on the standardization, modularization and open development of controller technology. Companies that started earlier in Europe and the United States are mainly represented by Delta Tau, AEROTECH and Trio, while the high-end controller technology in the Asia Pacific region is mainly in the hands of Japanese companies such as Yaskawa, Mitsubishi and OMRON. OMRON has introduced NX7 intelligent controller product by combining AI / IOT technology, which can monitor abnormal state of production process and realize predictive maintenance, which provides a new idea for the development of motion controller [1].

2. Motion Controller Based on Loongson

2.1. The hardware platform based on Loongson 3A3000

Loongson series processor is a general domestic processor developed by the Institute of computing technology, Chinese Academy of Sciences. It uses MIPS instruction system. Loongson 3A3000 is designed based on 3A2000, which greatly improves the buffer capacity and improves the chip frequency under the new technology. At the same time, the chip substrate bias adjustment support is added to better balance the contradiction between performance and power consumption, and broaden the applicability of the chip; in addition, it continues to maintain the forward compatibility of chip packaging pins, which can directly replace the original Loongson 3A series chips, and upgrade the BIOS and kernel, so that users can get a better experience[2]. This paper will use Loongson 3A3000 processor to realize the EtherCAT master.

2.2. Real time operating system of Loongson motion controller

At present, the commercial Loongson processor in the market is based on the Linux kernel operating system, but the Linux system is a time-sharing operating system, which can not meet the real-time requirements in motion control. Therefore, it is necessary to optimize the real-time performance of Linux and transform time-sharing operating system into real-time operating system. At present, there are two ways to improve the real-time performance of Linux system. One is to use dual core structure, that is, to embed a tiny, real-time kernel in the kernel of Linux operating system and transform it into a heterogeneous system with dual cores. The other is to directly modify the Linux kernel process, such as increasing kernel preemption, shortening the interrupt delay and so on, the representative of which is RT-Preempt [3]. As a real-time preemptive patch, RT-Preempt is updated and released together with Linux mainline kernel. It supports multiple architectures including MIPS. Although the response time of RT-Preempt is lower than that of VxWorks and RTLinux, RT-Preempt is constantly updated with the support of OSADL and Linux community, and the patch content will be gradually incorporated into the mainline kernel of the new version [4]. The comparison of various RTOS is shown in table 1. Based on the comparative analysis, it is appropriate to select RT-Preempt patch for real-time transformation of Linux system under MIPS architecture.

Table 1. Comparison of various RTOS.

RTOS	Advantage	Disadvantage
VxWorks	Excellent performance.	Expensive.
	Strong tailorability and rich interface resource.	Not easy to obtain.
RTAI	Fast response.	POSIX is not supported.
	More users and community support.	MIPS architecture is not supported.
RTLinux/GPL	Open source.	Update stopped
	Excellent performance.	
RT-Preempt	Strong community support.	The real-time performance is slightly weak
	Support a large number of platforms. Support POSIX and strong portability.	

2.3. Communication module of motion controller based on EtherCAT

The real-time Ethernet EtherCAT protocol is used in the communication module of motion controller. This protocol was proposed and implemented by Beckhoff company in the early 21st century, and became an international standard in 2005. Its main features are as follows [5]:

(1) It has wide applicability. There is no special requirement for the selection of master station computer. Any processor with commercial Ethernet controller can be used as master station.

(2) Fully compliant with Ethernet standards. The EtherCAT data frame is modified on the basis of the original Ethernet data frame, using a special type identification, so the protocol also achieves compatibility.

(3) The topology is flexible. Topology can be any shape, but whatever it is, it is logically circular.

(4) High efficiency. EtherCAT makes full use of the transmission bandwidth of Ethernet to exchange user data. It can complete the transmission and exchange of 1486 byte data frame in 300us, which is completely equivalent to 12000 digital input or output.

(5) The refresh cycle is short. Only 30us is needed to refresh 1000 digital I/O, and 100 I/O frames with an average of 8 bytes can be controlled within 100us.

(6) Good synchronization. Using the distributed clocks distributed on each slave node of EtherCAT, the clock synchronization accuracy between slave devices can be less than 1us.

3. Implementation of the System

3.1. The principle of RT-Preempt real-time patch

The real-time preemption patch of RT-Preempt is used to improve the real-time performance of standard Linux kernel by using interrupt threading, preemption of critical area and high-precision always.

Interrupt threading technology is to change the interrupt service program into a thread that can be scheduled by the operating system, and assign priority to the interrupt service program, so as to reduce the operation of closing interrupt, so as to avoid the situation that the real-time task can't be scheduled.

The standard Linux kernel does not support preemption. The kernel uses spin lock and large kernel lock to ensure data consistency. High-priority tasks can preempt the critical area. If a low-priority task is holding an rt_mutex lock at this time, a high-priority task will be added to rt_mutex wait_list(the wait_list is a priority queue of the lock) for the release of the lock [6].

The real-time system needs a high-precision clock system, and standard Linux uses jiffies to record the total number of clock interruptions from system startup to the present. So the real-time patch designs a new clock for the kernel management system [7].

3.2. Design of communication module based on EtherCAT protocol

For the communication management layer, we use the EtherCAT communication protocol. The EtherCAT open source master stations announced on the Internet include SOME and IgH EtherCAT Master. SOME supports Windows and Linux2.6 platforms, and the SOME function is relatively simple; IgH EtherCAT Master is based on the Linux platform. It has been nearly 10 years since its release and has been updated many times. The function is relatively complete, and it also supports various real-time extensions such as RTAI, RT-Preempt, Xenomai, and supports DC distributed clocks, and supports multiple communication protocols such as COE, SOE, EOE, etc. In comparison, the IgH master station code is more mature and more suitable for the development of EtherCAT master station [8]. But IgH EtherCAT open source Master does not support Loongson's network card.

In this paper, another idea is adopted. the EtherCAT communication protocol stack module is moved to the user space. And the user space communication module is modified based on the existing open source EtherCAT master station code to realize the user mode EtherCAT master station protocol in Loongson motion controller. And adopts the idea of hardware abstraction and software encapsulation to shield the specific implementation details of the bottom layer and provide a public access interface to the upper layer. For interoperability between functional modules and communication services, we can use shared memory, semaphore mechanism, pipeline communication and message queue mechanism, etc. Since we implement the EtherCAT master station communication protocol in user mode, we capture the data frame before entering the kernel network protocol stack, and then send the captured data frame directly to the EtherCAT protocol stack in user space. The specific architecture is shown in Fig. 1 [9].

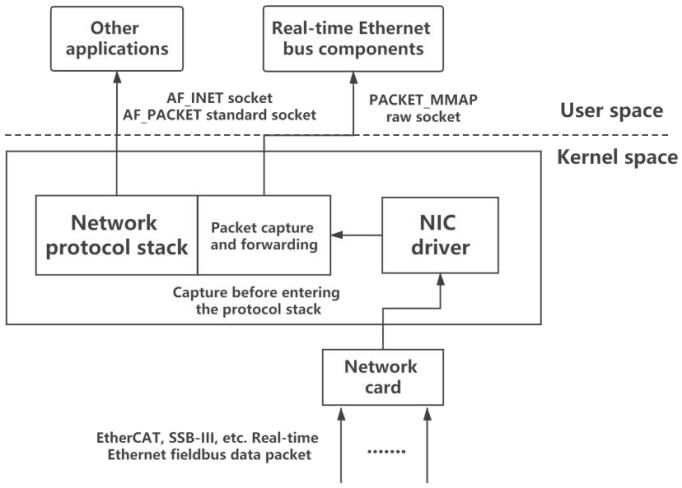

Fig. 1. Packet capture architecture diagram

The specific capture of data packets is through the memory mapping mmap system call to map the user mode address space to the kernel mode. User mode components can directly access the data in the kernel mode buffer and use a polling mechanism. When the data is ready, the user mode component can be used directly, reducing the copy of the data in the memory. In addition, socket programming uses raw sockets. Ordinary sockets can only work at the transport layer and provide services for the application layer. We can use raw sockets to work below the transport layer, and raw sockets can send and receive data frames at the data link layer. The specific data flow is shown in Fig. 2.

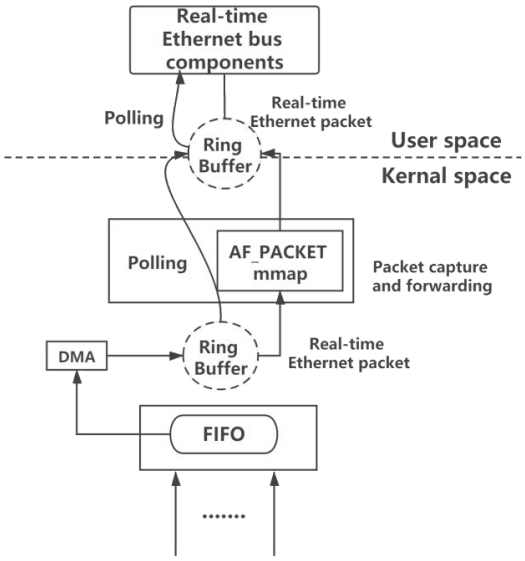

Fig. 2. Packet flow diagram

After we transfer the communication module from the kernel space to the user space, the basic structure of the motion controller is determined. All the functional modules of the motion controller based on Loongson are moved to the user mode. In the above communication module architecture diagram, we use some kernel functions and sockets. According to these kernel functions and sockets, we have completed the data frame capture, and then the data frame can be processed according to the EtherCAT protocol stack. In this process, we need to build an environment and design sending and receiving experiments to ensure that the capture of data frames is successful. When using the EtherCAT protocol to process data frames, we also need to use the PACKET_MMAP mechanism to capture the data frame packet and verify that the format of our data frame processing is also correct. After the completion of data frame composition, analysis and transceiver module, the design of data interaction between communication module and motion control module should be considered, which involves the design of synchronization and inter process communication, so as to make the modules consistent, calculate load balance and reduce the data loss rate.

4. Experiment and Verification

4.1. Set up the experimental environment

Install the VMvare virtual machine in the host, and install the Linux operating system in the virtual machine. Three network working modes are provided in VMWare, they are: Bridged, NAT and Host-Only. In this experiment, NAT mode is adopted, which enables virtual system to access the Internet through the network where the host machine is located. The biggest advantage of using the NAT mode is that the virtual system is easy to access the Internet and does not require any other configuration. The disadvantage is that the configuration information cannot be manually modified because the TCP/IP configuration information of the virtual system is provided by the DHCP server of the VMnet8 virtual network.

4.2. Frame and send

We framing in the user mode according to the EtherCAT data frame format, and then directly send the data frame to the network card through the PACKET_MMAP mechanism, skipping the core network protocol stack. The main steps are as follows:

(1) Obtain local network card information and MAC address.

(2) Find interface index from interface name and store index in struct sockaddr_ll device.

(3) Framing. Set the source MAC address, set the destination MAC address to the broadcast address, and set the EtherCAT data header and body.

(4) Create socket and send data packet.

The key code is as follows:

Table 2. The main code.

Program Create and send EtherCAT frame

```
uint8_t data[IP_MAXPACKET];
uint8_t src_mac[6];
uint8_t dst_mac[6];
// Set destination MAC address. The broadcast address. dst_mac = {0xff, 0xff, 0xff, 0xff, 0xff, 0xff}
// Framing EtherCAT data frame. for example data = {0x0e, 0x10, 0x02, 0x00, 0x00}
// Submit request for a raw socket descriptor.
if ((sd = socket (PF_PACKET, SOCK_RAW, htons (ETH_P_ALL))) < 0)
{
   perror ("socket() failed ");
   exit (EXIT_FAILURE);
}
// Send ethernet frame to socket.
if ((bytes = sendto (sd, ether_frame, frame_length, 0, (struct sockaddr *) &device, sizeof (device))) <= 0)
{
   error ("sendto() failed");
   exit (EXIT_FAILURE);
}
```

4.3. Receive and parse

In the above master station sending process, we form an EtherCAT data frame in the user mode and sent it directly to the network card. The receiving process of the master station is opposite to the sending process. We need to capture the data frame from the network card and send it directly to the user mode. Then we parse the data frame format according to the EtherCAT data frame in the user mode to prove that our master station sends and receives data frames is feasible. The main steps are as follows:

(1) Open the socket.

(2) Set the ring buffer of mmap.

(3) Map the ring buffer to user space.

(4) Polling. Wait for new data packets to be captured at the bottom layer.

The key code is as follows:

Table 3. The main code.

Program Receive and parse EtherCAT frame

```
// Setup the fd for mmap() ring buffer
// Set block_size, frame_size, block_nr, frame_nr
if ( (setsockopt(fd, SOL_PACKET, PACKET_RX_RING, (char *)&req, sizeof(req))) != 0 )
{
    perror("setsockopt()");
    close(fd);
    return 1;
};

// Polling
for(;;)
{
    while(*(unsigned long*)ring[i].iov_base)
    {
      struct tpacket_hdr *h=ring[i].iov_base;
      struct sockaddr_ll *sll=(void *)h + TPACKET_ALIGN(sizeof(*h));
      unsigned char *bp=(unsigned char *)h + h->tp_mac;
      // Analyze the data frame
      // Remove the data link layer header, and get the EtherCAT data
      uint8_t temp[1024] ={0};
      memcpy(temp,(char*)h+h->tp_mac,h->tp_len);
      // Deal the EtherCAT data
      // Tell the kernel this packet is done with
      h->tp_status=0;
      i=(i==req.tp_frame_nr-1) ? 0 : i+1;
    }
    // Sleep when nothings happening
    pfd.fd=fd;
    pfd.events=POLLIN|POLLERR;
    pfd.revents=0;
    poll(&pfd, 1, -1);
}
```

4.4. Verification

We use wireshark to capture data packets to verify whether the data frame sending and receiving experiment is correct. We use wireshark to monitor the network card through which the data frame passes, and the capture mode of wireshark is set to promiscuous mode. In addition, our network card needs to support promiscuous mode, which is a driving mode that allows the network card to view all data packets flowing through the network line. Therefore, even if it is a useless packet, the network card will not choose to discard it, and will still be passed to the CPU for processing. This way we will not worry about losing any critical information.

We framing in the user mode of the master station, and directly sent to the network card, and use wireshark to capture this data frame. In addition, the receiving and sending of the data frame of the master station is the opposite process. And the master station can also capture the data frame by turning on the receiving function.

```
lizeyang@lizeyang-virtual-machine:~/wch/send$ cc send_ethercat.c
lizeyang@lizeyang-virtual-machine:~/wch/send$ sudo ./a.out
MAC address for interface ens33 is 00:0c:29:84:11:87
Index for interface ens33 is 2
send num=28,read num=28
lizeyang@lizeyang-virtual-machine:~/wch/send$
```

Fig. 3. Frame and send EtherCAT frame

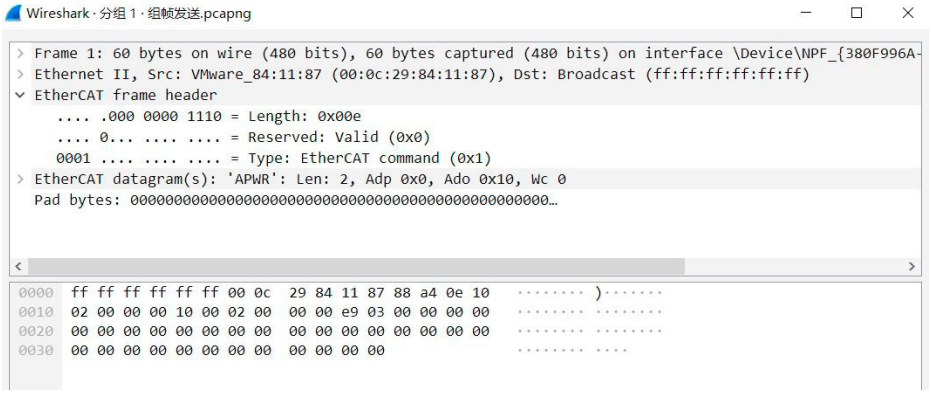

Fig. 4. Use wireshark to capture frame

```
lizeyang@lizeyang-virtual-machine:~/wch/recive$ cc recive_ethercat.c
lizeyang@lizeyang-virtual-machine:~/wch/recive$ sudo ./a.out
1602674415.126109: if2 > 14 bytes
0e 10 02 00 00 00 10 00 02 00 00 00 e9 03
^Crecieved 1 packets, dropped 0
lizeyang@lizeyang-virtual-machine:~/wch/recive$
```

Fig. 5. Recive the EtherCAT frame

It can be seen from the data frames captured by wireshark and the receiving function of the master station that the sending and receiving functions of the data frame are correct. It is feasible for the Loongson motion controller communication module to adopt this scheme.

5. Conclusion

As the current EtherCAT open source master station does not support Loongson network card, the network layer and transport layer protocol of the original network protocol stack do not support real-time Ethernet EtherCAT data frame, and the data frame will be directly discarded without processing. This article provides another way to move the EtherCAT communication protocol stack to the user space. For the sending and receiving of EtherCAT data frames, the system call MMAP mechanism must be used to capture data frames in the network card driver to skip the network protocol stack in the kernel space, and then send the captured data frame directly to the user mode EtherCAT communication protocol stack for processing.

After we moved the original EtherCAT protocol stack in the kernel space to the user space, the motion control module, technology package and communication module are all in the user space, reducing system calls and efficient data frame processing. Another main reason is that the kernel space does not support floating-point arithmetic, so our data processing will not be restricted, which further improves the flexibility of the motion controller. Finally, the implementation of the user mode EtherCAT master protocol which needing to learn from the current open source EtherCAT master code, the implementation of the data interaction between the motion control module and the communication module, need to be completed in the future work.

Acknowledgment

This work was supported by National Key R & D Program of China (No.2018YFB1308803), the project of "Prospering Liaoning Talents Program" (XLYC1807195) of Liaoning Province and funded by China Postdoctoral Science Foundation, 2018M641729.

References

1. Qiang Li. Design and Implementation of Multi-axis Motion Control System Based on Industrial Ethernet EtherCAT [D]. Guangdong University of Technology,2019.
2. http://www.loongson.cn/news/company/472.html.
3. Beibei Li,Yong Luan,Chao Wang,Zhe Wang,Liaomo Zheng. Construction of Embedded Real-time EtherCAT Master Station Based on AM3358 Processor [J]. Modular machine tool and automatic processing technology,2015(05):1-5.
4. Pu Wang,Qing Zhou. Real-time optimization and reliability design of embedded Linux based on Loongson 1E [J].Microelectronics and Computer,2019,36(11):11-15.
5. Yu Liang. Explore the characteristics and applications of EtherCAT industrial Ethernet technology [J]. Science and Technology Innovation, 2019(03):99-100.
6. Zhangjin Wu. Research and practice of Linux real-time preemptive patch [D]. Lanzhou University,2010.
7. Ji Liu. Tickless Technology Research and Its Implementation in Embedded System [D]. University of Electronic Science and Technology,2009.
8. Yiren Tu. Design of EtherCAT master station based on IGH open source framework [J]. Automation application,2020(05):65-66+69.
9. Beibei Li,Hu Lin,Liaomo Zheng,Shujie Sun,Zhenyu Yin. An Open CNC System based on EtherCAT Network[A]. IEEE Beijing Section,Global Union Academy of Science and Technology,Chongqing Global Union Academy of Science and Technology.Proceedings of 2016 IEEE Advanced Information Management,Communicates,Electronic and Automation Control Conference(IMCEC 2016)[C].IEEE Beijing Section, Global Union Academy of Science and Technology,Chongqing Global Union Academy of Science and Technology:IEEE BEIJING SECTION(Transnational Institute of Electrical and Electronics Engineers Beijing Branch),2016:7.